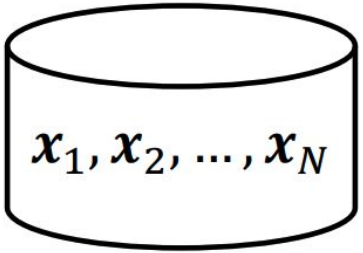
Approximate Nearest Neighbor Search Using Neighborhood Graphs

Presented by Minh Phan

Nearest Neighbor Search

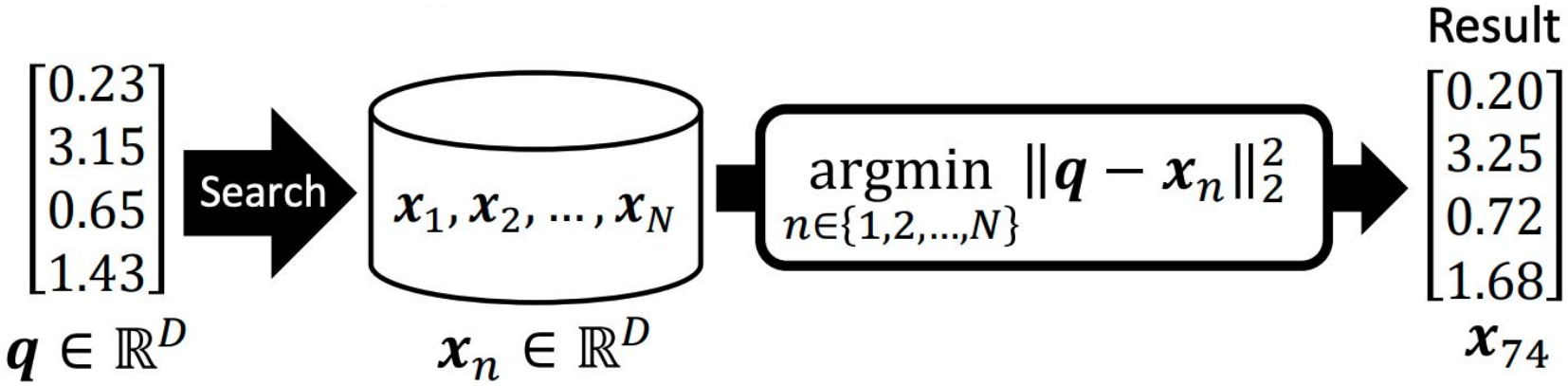


$\boldsymbol{x}_n \in \mathbb{R}^D$

- **N** vectors with dimension **D**

Slide adapted from Yusuke Matsui (CVPR 2020)

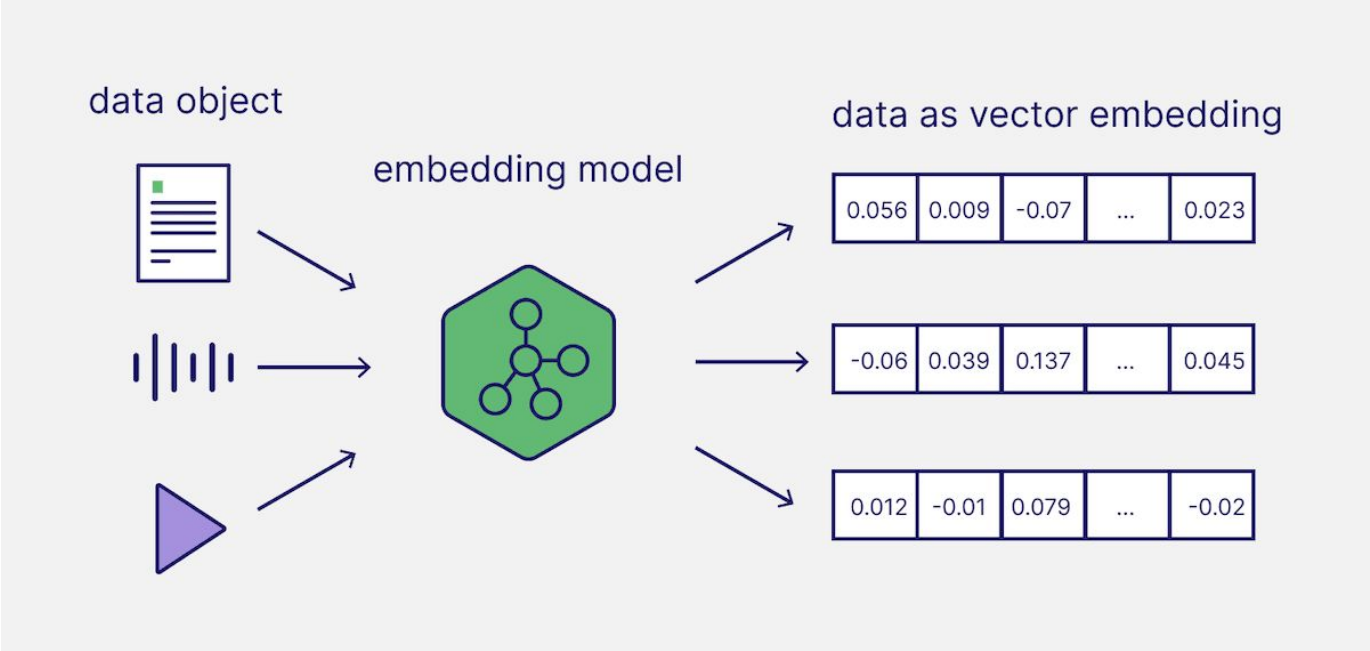
Nearest Neighbor Search



- **N** vectors with dimension **D**
- Given a vector query **q**, find its **k** nearest neighbors

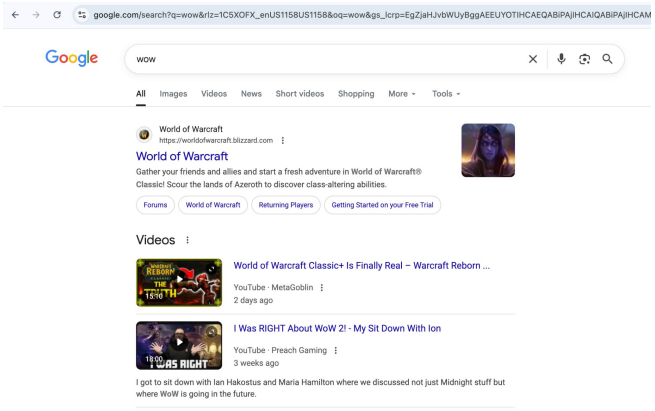
Slide adapted from Yusuke Matsui (CVPR 2020)

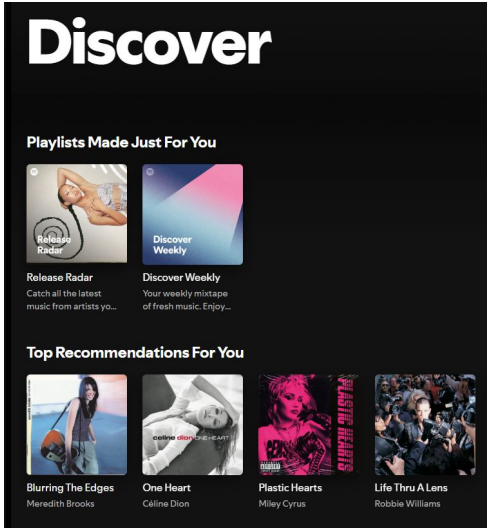
Embedding models



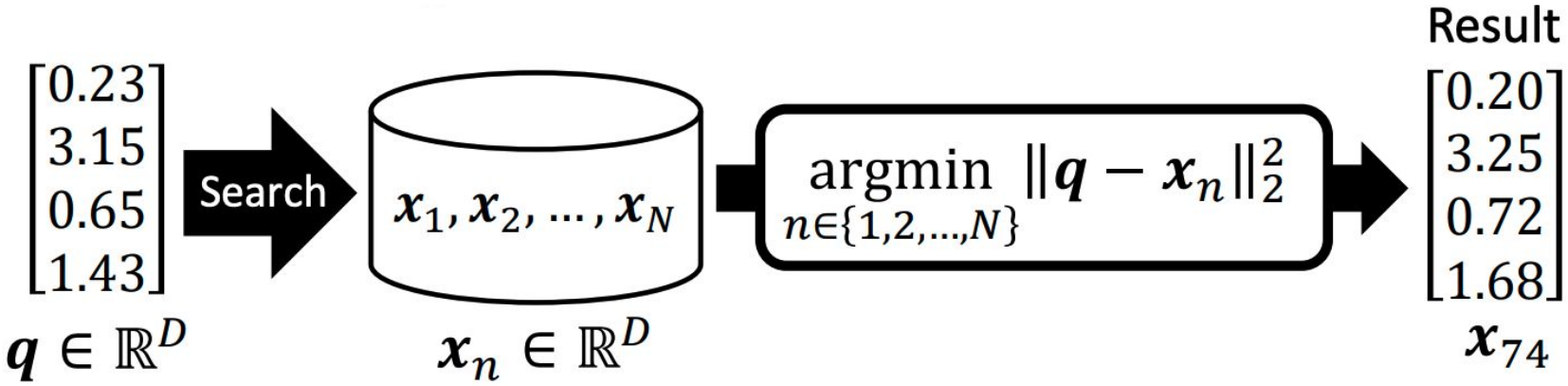
Applications

- ➢ Semantic Search
- ➢ Recommendations
- ➢ Image/Audio Search
- ➢ Retrieval-Augmented Generation (RAG)





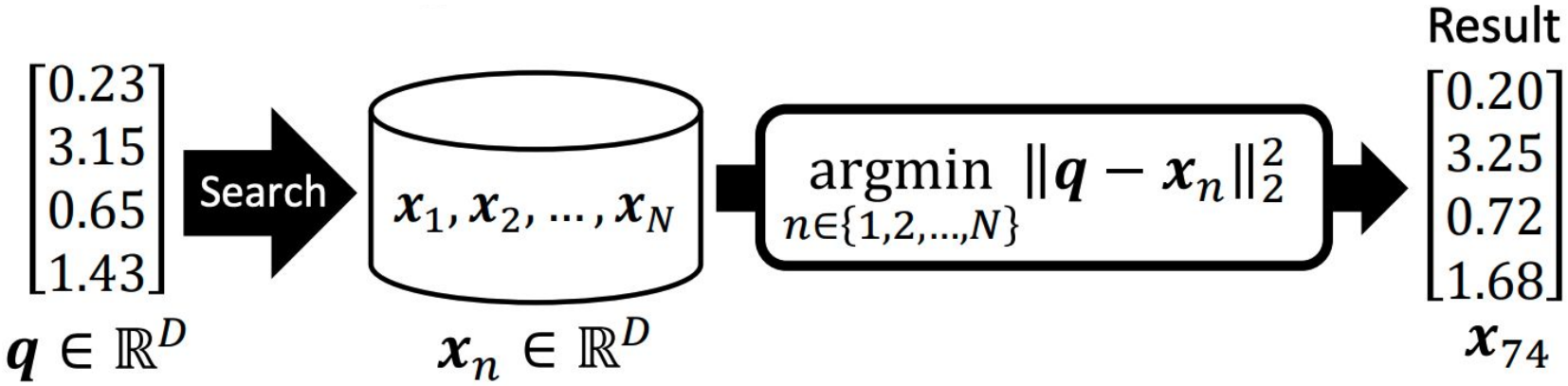
Nearest Neighbor Search



- **N** vectors with dimension **D**
- Given a vector query **q**, find its **k** nearest neighbors
- Bruteforce search complexity: **O(ND)** → too slow at scale

Slide adapted from Yusuke Matsui (CVPR 2020)

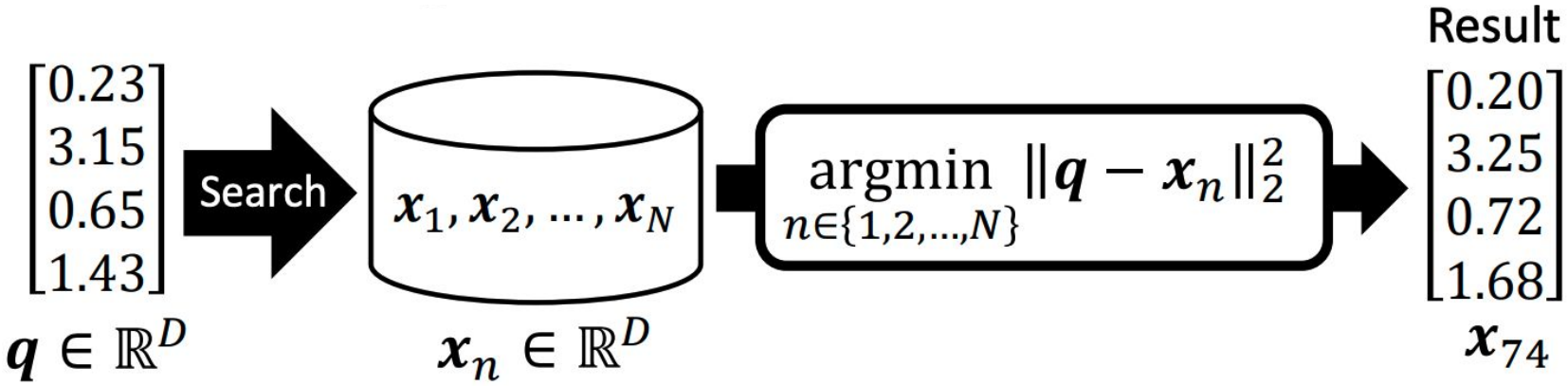
Approximate Nearest Neighbor Search



- Build an index to search quickly (sublinear complexity) with high **recall**
- Tradeoffs between **runtime** ↔ **accuracy (recall)** ↔ **memory usage**
- Makes billion scale search practical

Slide adapted from Yusuke Matsui (CVPR 2020)

Approximate Nearest Neighbor Search

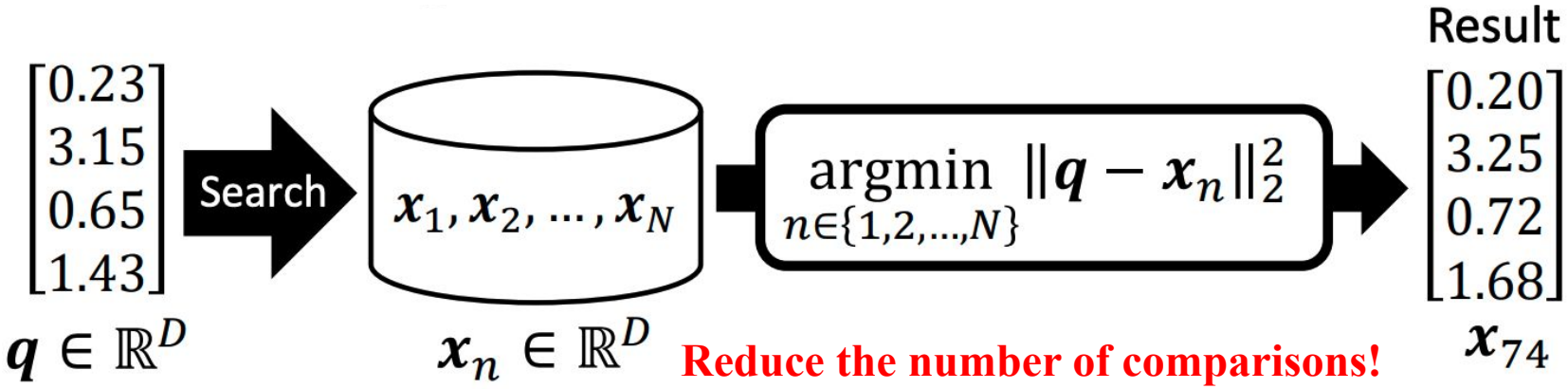


- Build an index to search quickly (sublinear complexity) with high **recall**
- Tradeoffs between **runtime** ↔ **accuracy (recall)** ↔ **memory usage**
- Makes billion scale search practical

R@k = # true results / k

Slide adapted from Yusuke Matsui (CVPR 2020)

Approximate Nearest Neighbor Search



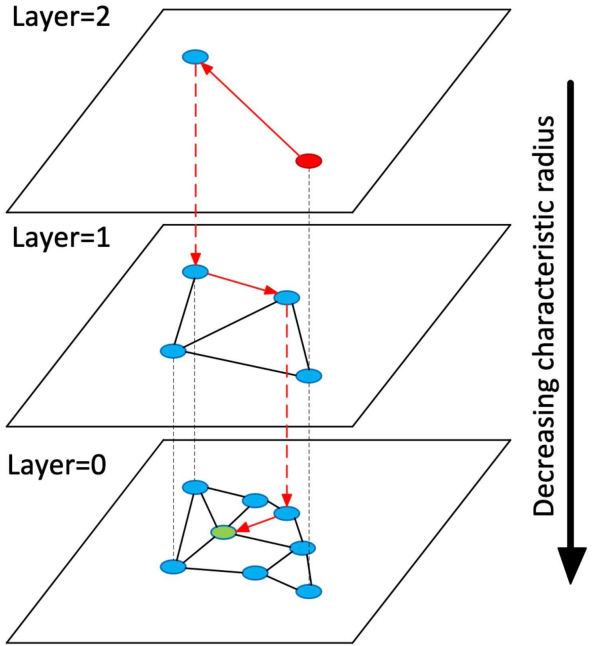
- Build an index to search quickly (sublinear complexity) with high **recall**
- Tradeoffs between **runtime** ↔ **accuracy (recall)** ↔ **memory usage**
- Makes billion scale search practical

R@k = # true results / k

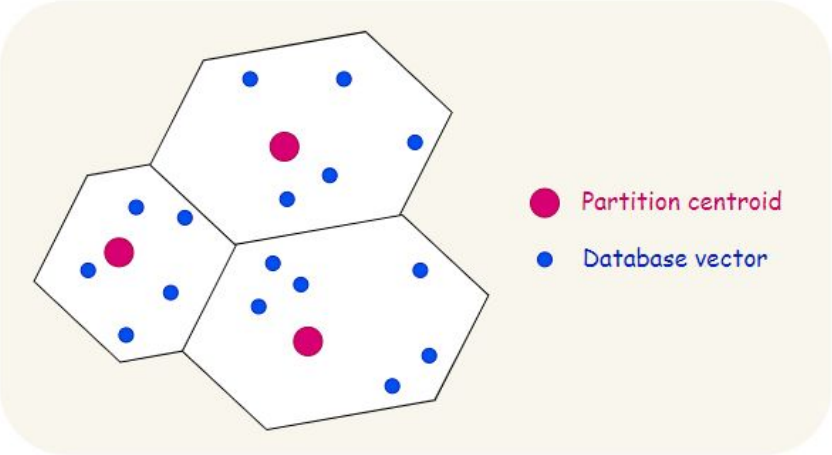
Slide adapted from Yusuke Matsui (CVPR 2020)

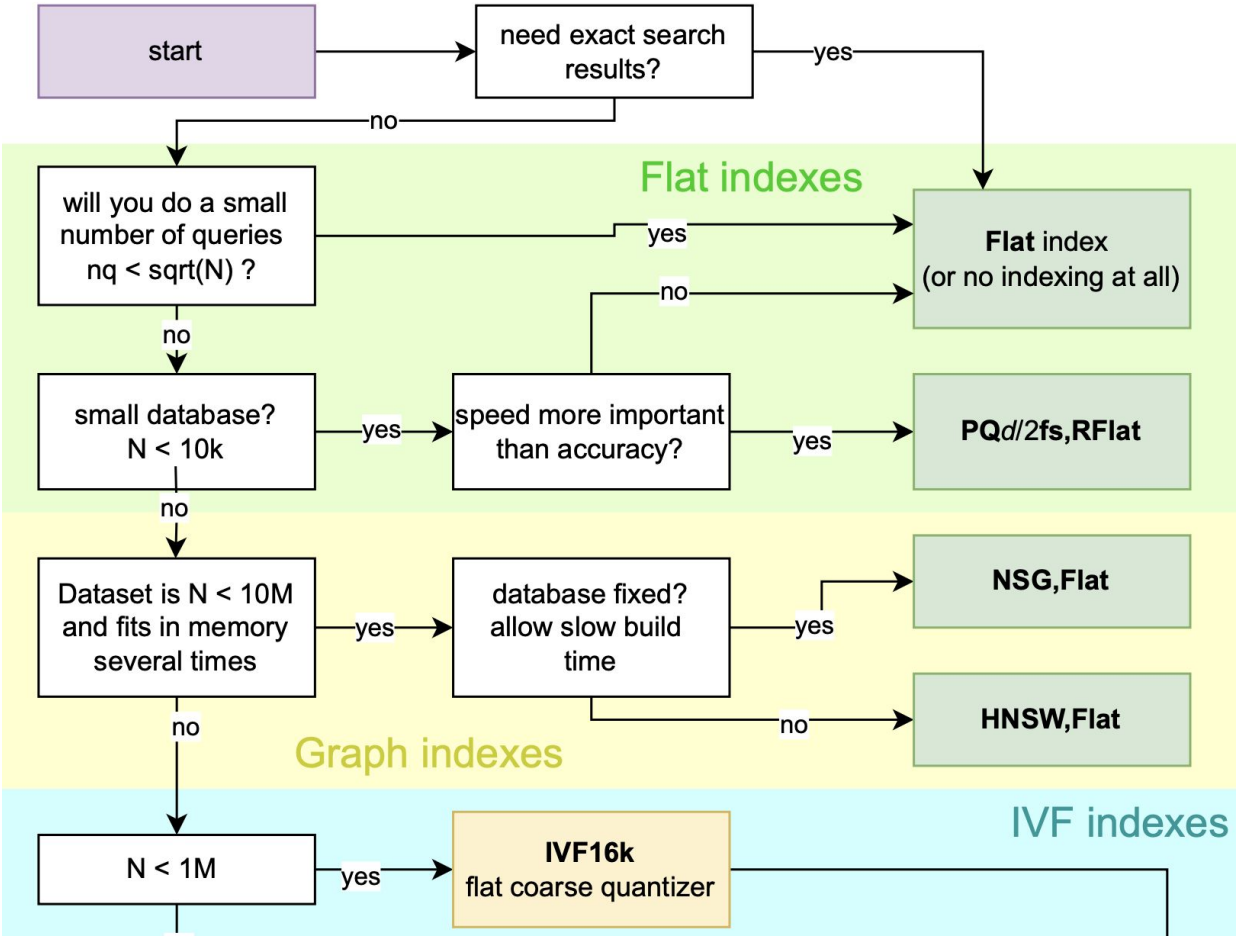
Vector Index

Graph

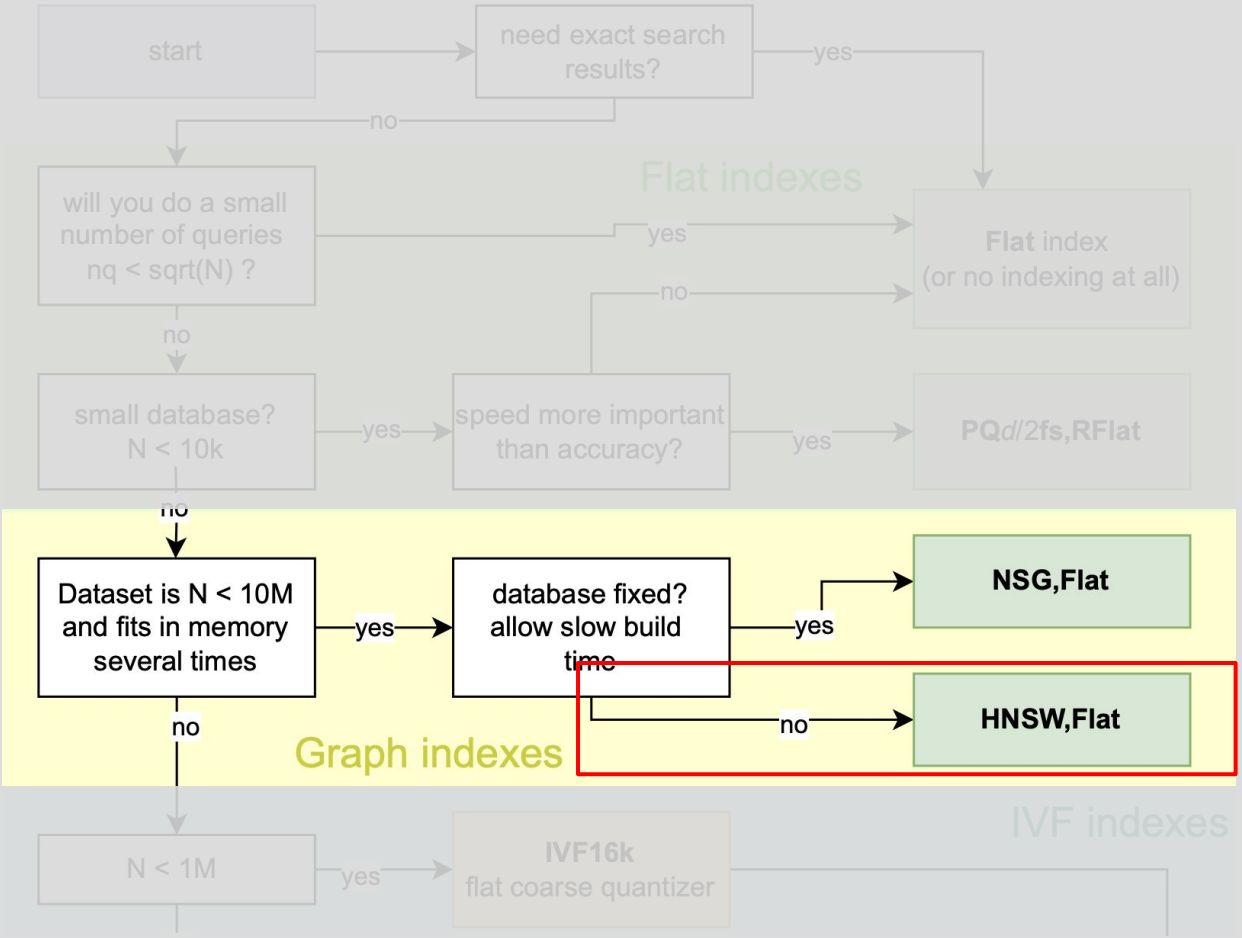


Clustering





The FAISS Library (2024)

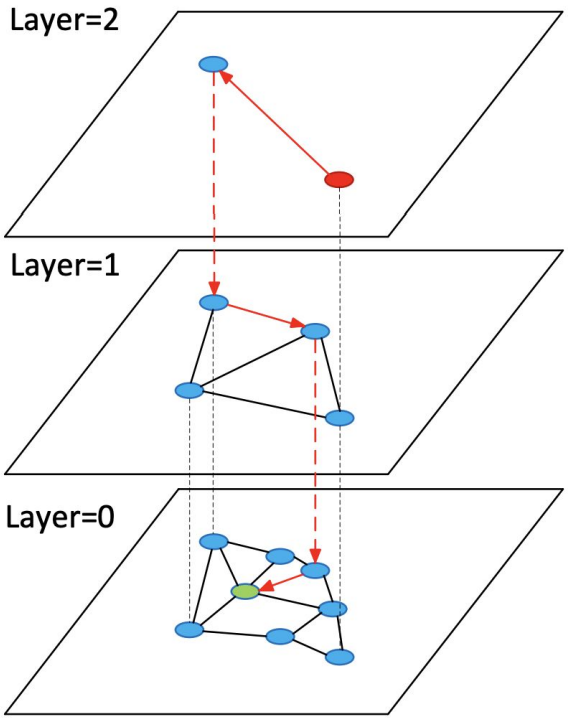


accurate, allow incremental update

The FAISS Library (2024)

Hierarchical Navigable Small World (HNSW)

- Pros: Extremely fast, high-recall search and supports easy incremental updates
- **Cons:** High memory usage - all vectors ideally need to reside in memory
- Each **node** is a vector
- **Edges** link nodes that are nearby, with a few long-range links
- **Search**: Starts from one entry point and gradually traverse toward nearer points

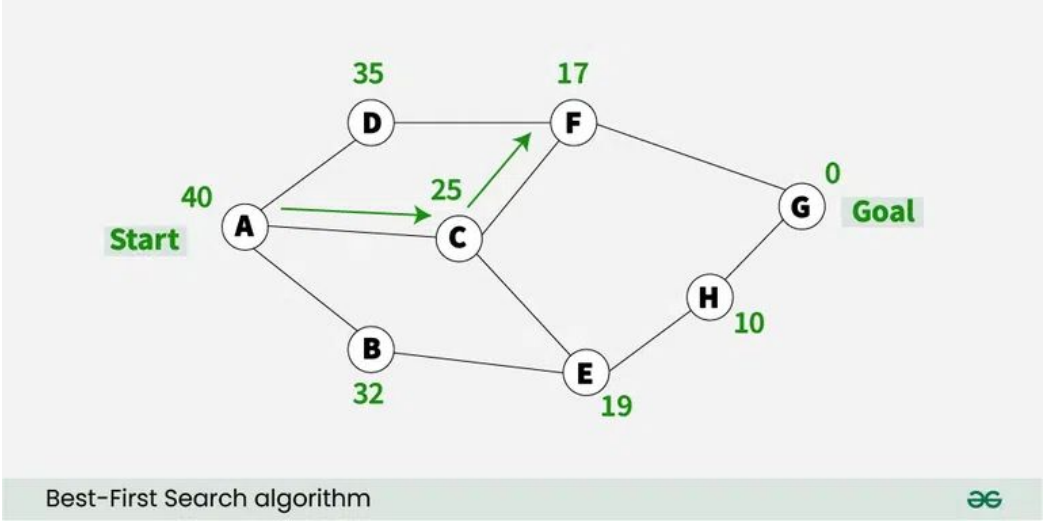


The idea behind graph index

Each node represents a vector and connected to some other nodes

We walk the graph, starting from one node and greedily moving to neighbors closer to the query

Greedy search explores only a small portion of the graph while still finding nodes close to the query

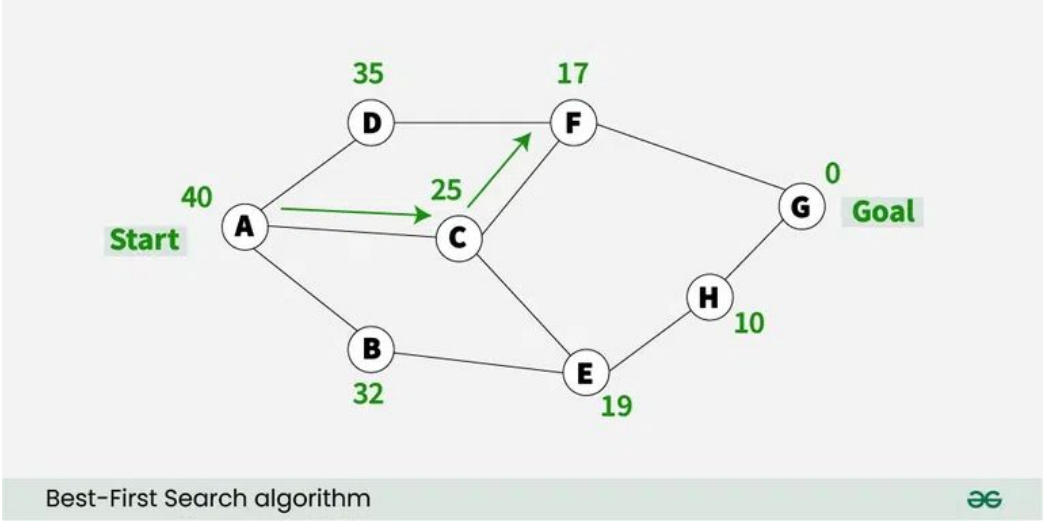


The idea behind graph index

Each node represents a vector and connected to some other nodes

We walk the graph, starting from one node and greedily moving to neighbors closer to the query

Greedy search explores only a small portion of the graph while still finding nodes close to the query



Will greedy search always guarantee nearest neighbor? Depends on graph!

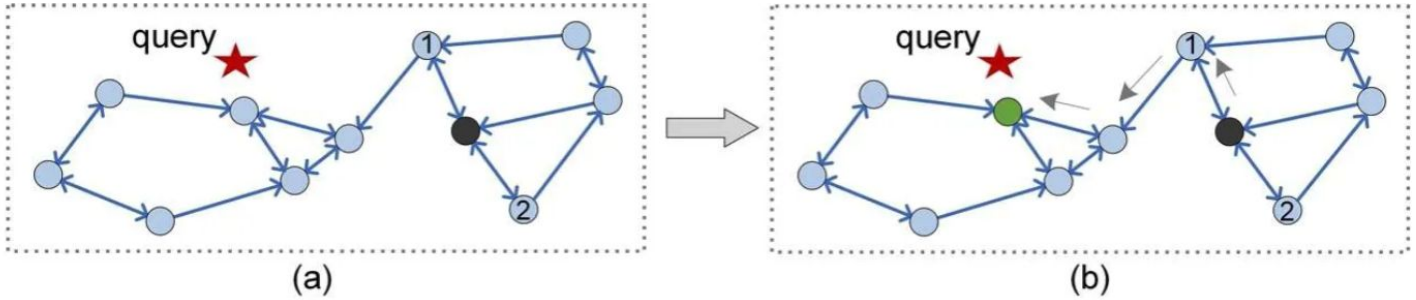


Figure 2: Routing procedure on a PG index (KNNG, k=2).

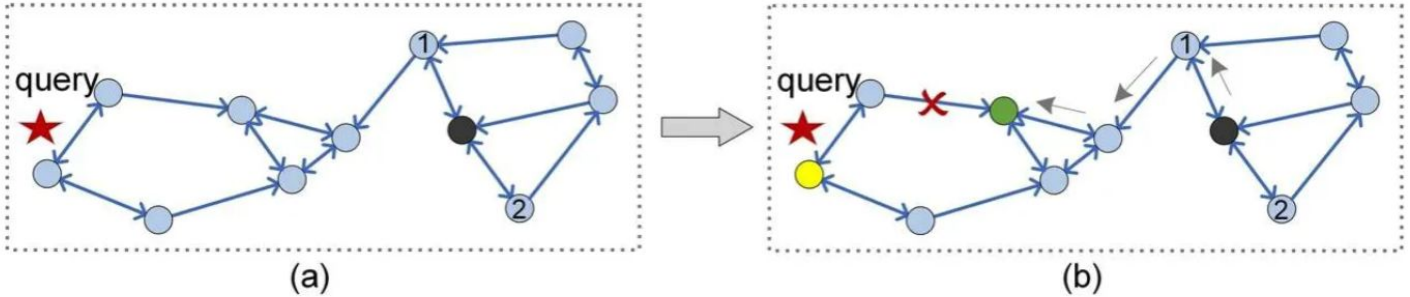


Figure 3: Routing starting from the black vertex cannot reach the yellow vertex.

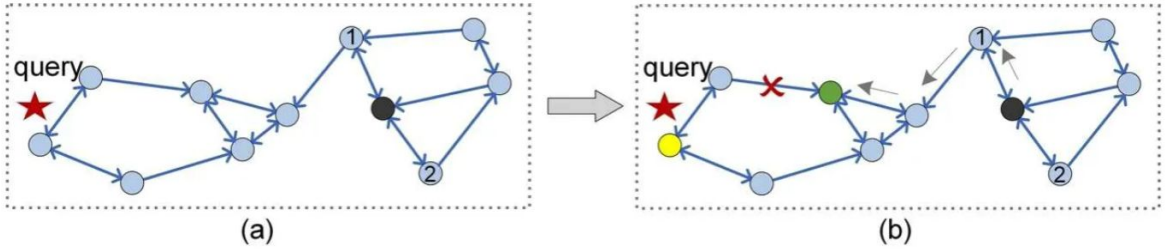


Figure 3: Routing starting from the black vertex cannot reach the yellow vertex.

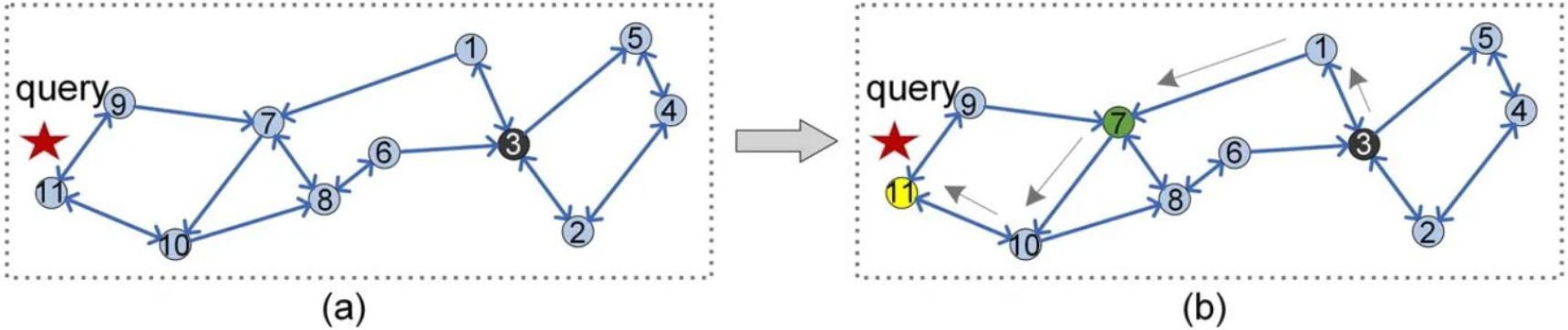
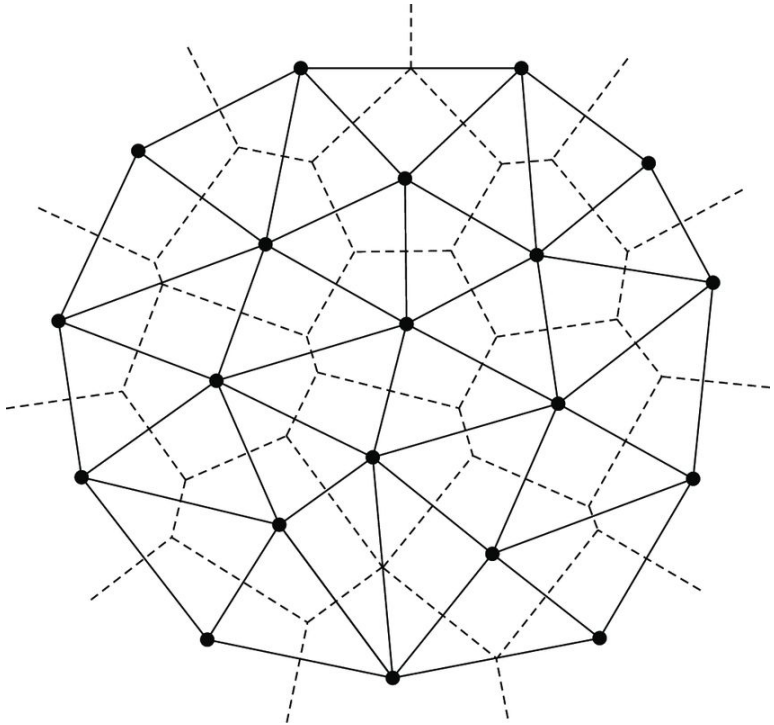


Figure 7: Routing procedure on a Relative Neighborhood Graph (RNG).

Theoretical Ideas - Delaunay Graph



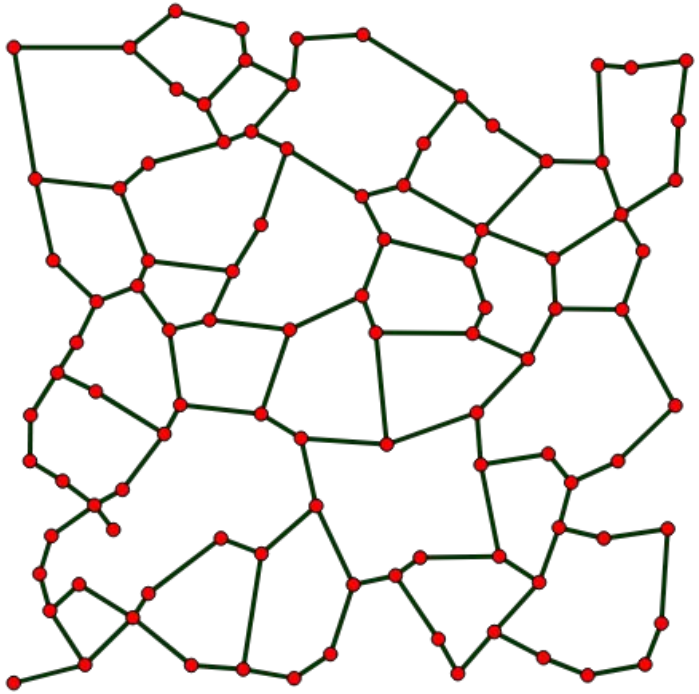
Delaunay Graph

"A perfect navigable graph"

Impossible to construct in practice!

$$O(n^{\lceil d/2 \rceil})$$

Theoretical Ideas - Relative Neighborhood Graph (RNG)

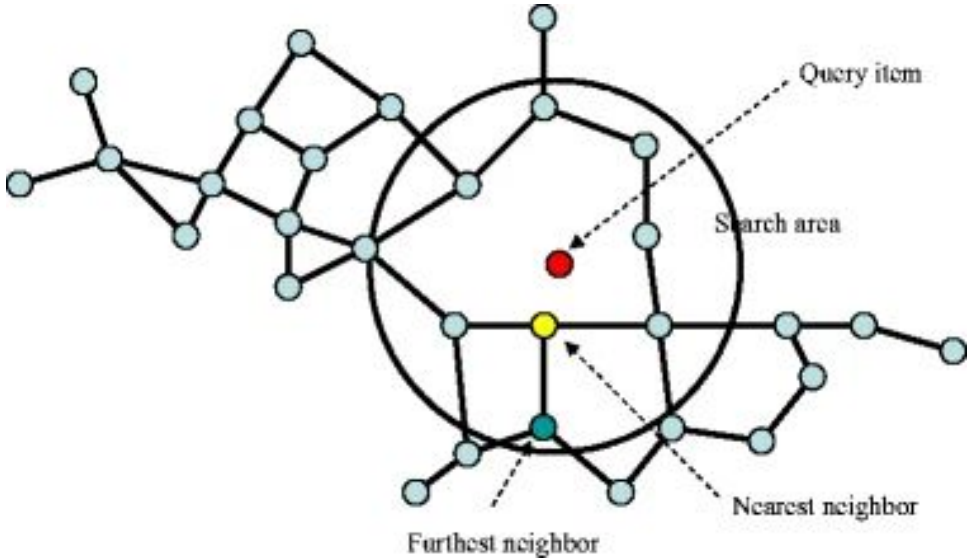


Relative Neighborhood Graph

Connect two nodes only if no other node is closer to both than they are to each other.

$$O(n^3)$$

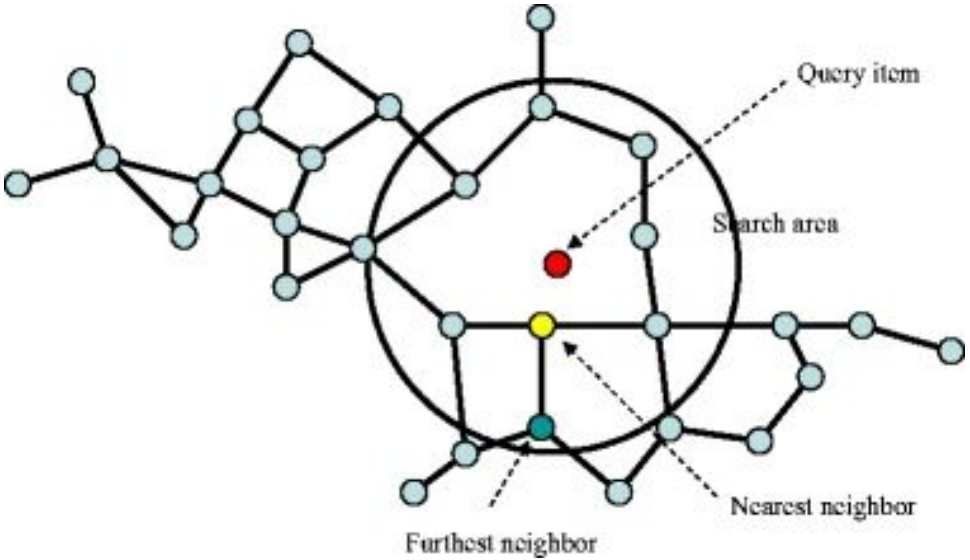
Theoretical Ideas - Constructing Relative Neighborhood Graph



- When inserting a node, find its **exact nearest neighbor** (O(N))
- Figure out the l**ocal neighborhood** (the circle)
- Ensure RNG property in that region O(k^3)

$$O(n^3)$$

Theoretical Ideas - Constructing Relative Neighborhood Graph



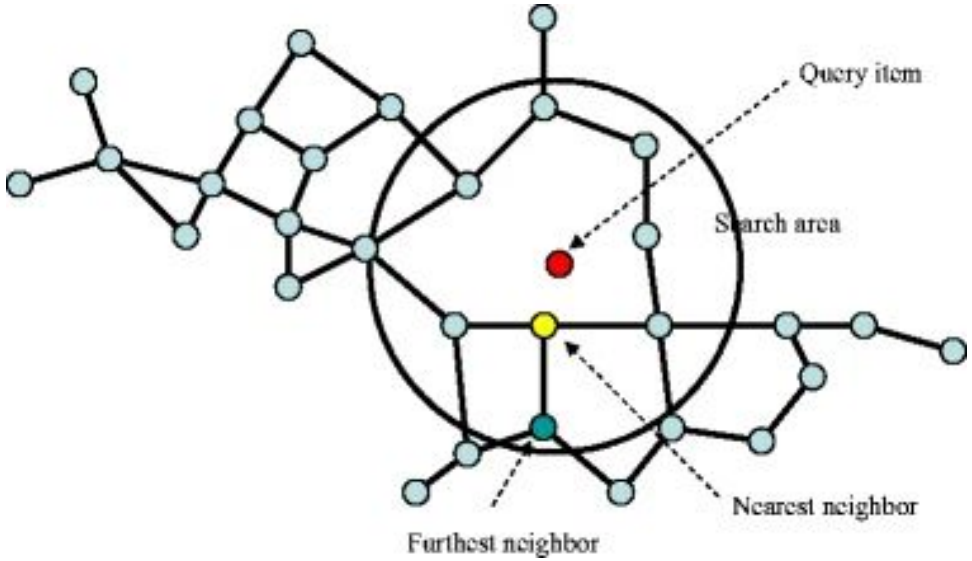
- When inserting a node, find its **exact nearest neighbor** (O(N))
- Figure out the **local neighborhood** (the circle).
- Ensure RNG property in that region O(k^3)

$$O(n^3)$$

Approximating a Relative Neighborhood Graph

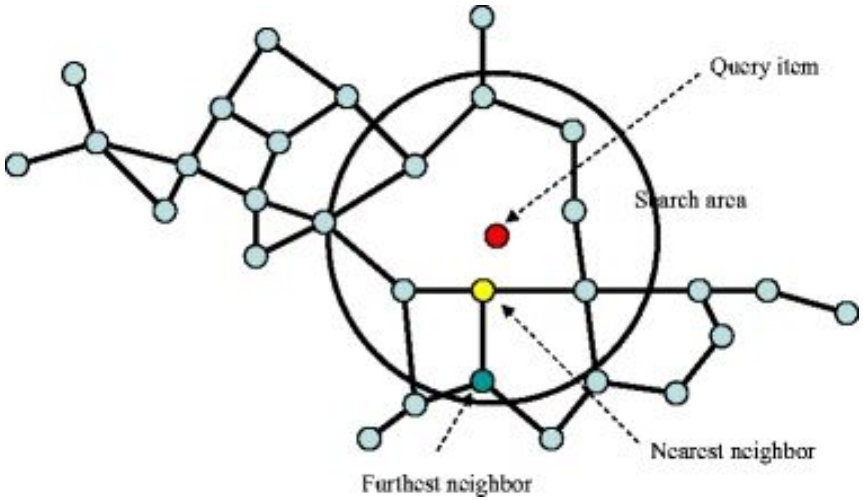
Still too expensive to construct!

Practical Ideas - Approximating Relative Neighborhood Graph



- When inserting a node **X**, find its approximate nearest neighbor **Y** via greedy search.
- Find the top k nearest neighbor to X (greedy search from Y). That's the local neighborhood for X.
- Add k new edges, ensure some Relative Neighborhood Graph property in that neighborhood.

Practical Ideas - K-Nearest Neighbor Graphs with RNG pruning



Algorithm 1: The General Graph Construction Framework(V, K)

Input: V is the vertex set and K is used to control the graph qualilty

Output: The Search Graph G

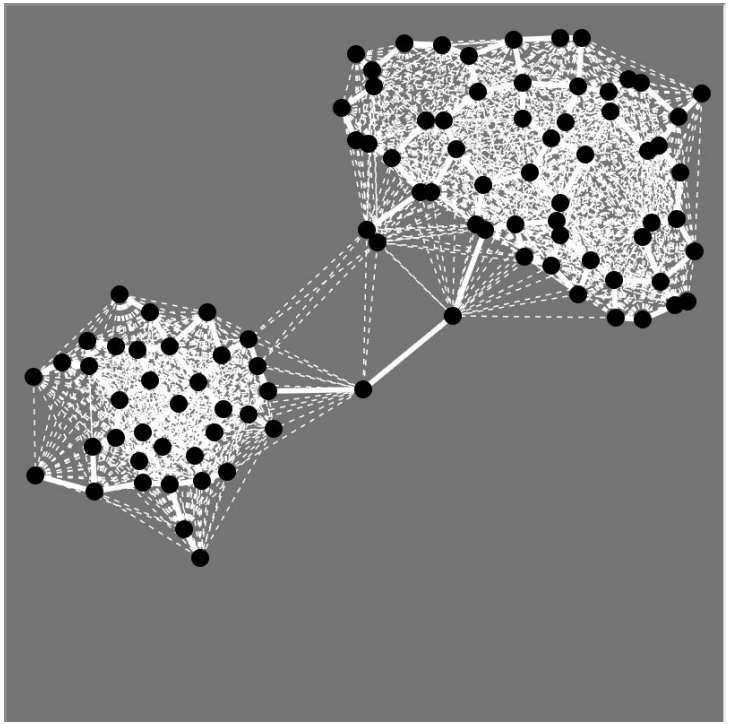
```
for each v ∈ V do
    Find the exact or approximate K nearest neighbors of v;
    Choose a subset C of these KNNs based on some specific heuristics;
    Add directed or bi-directional connections between v and every vertex in C;
    // update G
return G
```

Problems with kNN graphs (and RNG)

On clustered data

- With a bad starting point, you might be traversing too many nodes before reaching your destination.
- kNN methods approximate RNG, so the graph may not be complete!

Relative Nearest Neighborhood Graph on clustered data



https://luthert.web.illinois.edu/blog/posts/221.html

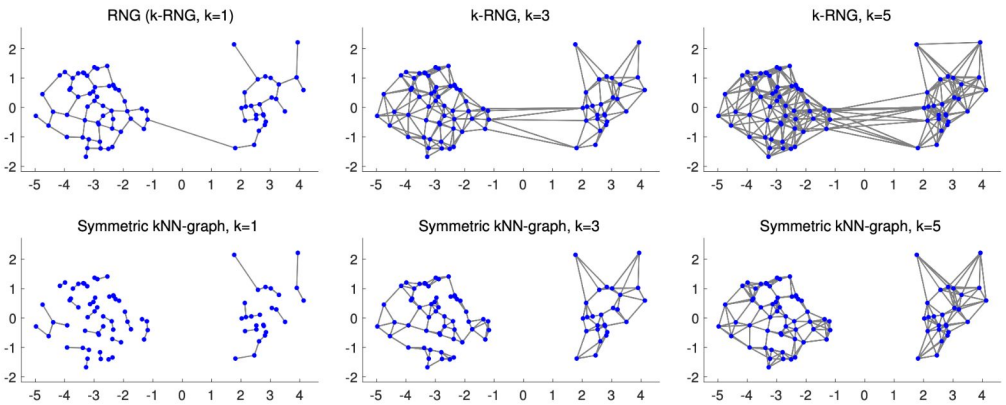


Figure 3: k-relative neighborhood graphs (1st row) and symmetric k-nearest neighbor graphs (2nd row) on 80 points from a mixture of two Gaussians. Note that as opposed to the k-NN graphs, the k-relative neighborhood graphs tend to have more connections between points from the different mixture components. In fact, a k-RNG is always connected (see Section 5.3). This might be desirable in some situations, but undesirable in others.

Lens Depth Function and k-Relative Neighborhood Graph: Versatile Tools for Ordinal Data Analysis (2017)

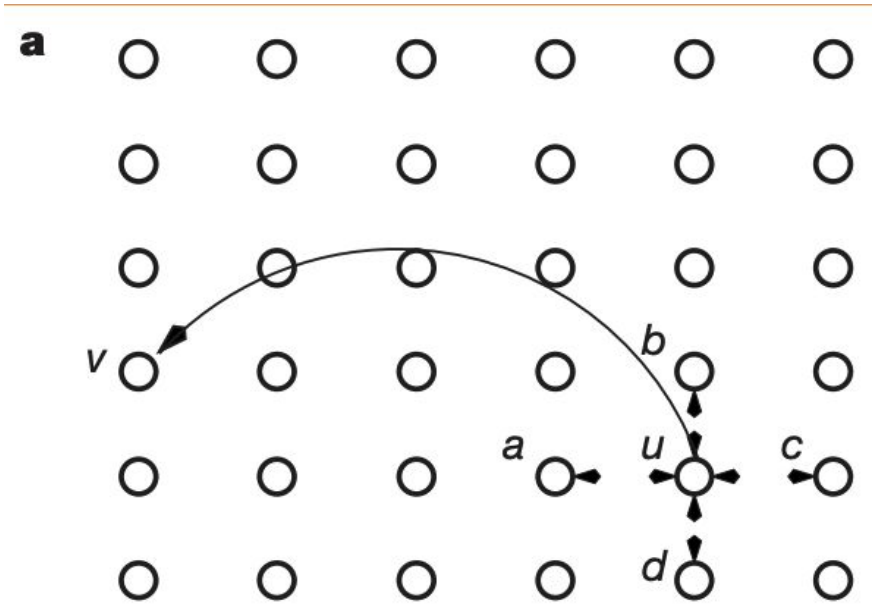
Approximations of Relative Neighborhood Graphs:

- Good **local neighborhood**, so if we start from the right neighborhood, we are likely to be able to greedy search to our destination.
- **Lack of global connectivity** → worst case we might have to traverse through all nodes!

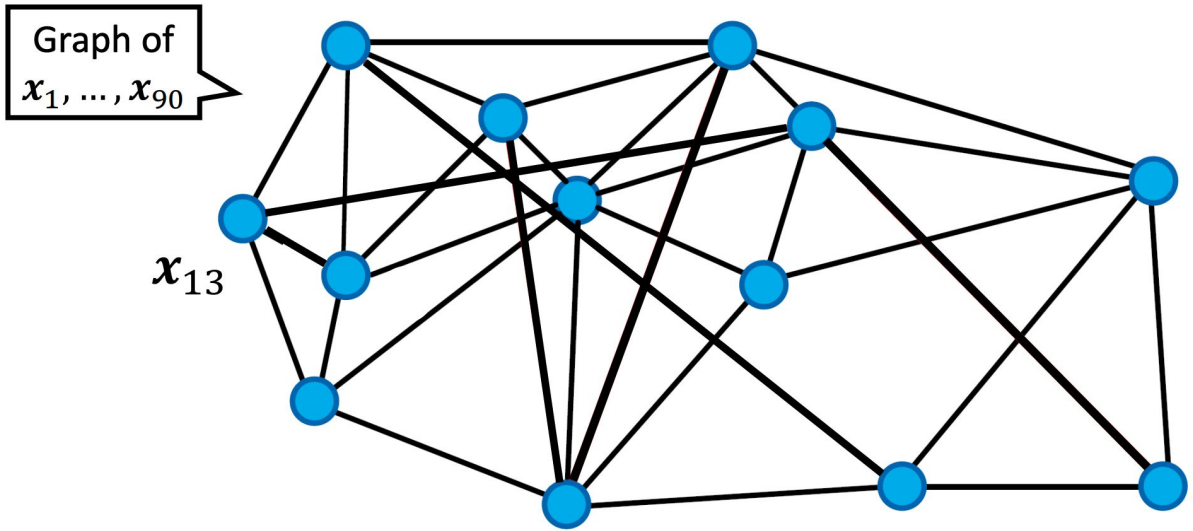
Navigation in a small world

In Kleinberg's small-world model, each node connects to its close neighbors plus a few long-range links chosen with a probability inversely proportional to distance.

These rare long links act as shortcuts, allowing greedy navigation to reach any target in about O(log^2(n)) steps.



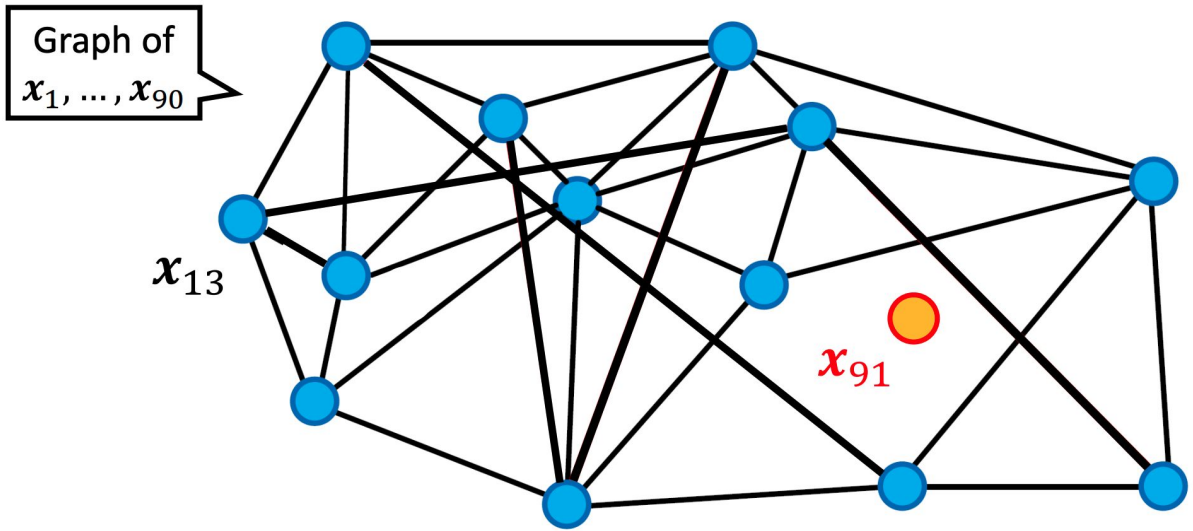
Navigable Small World (NSW)



- K-NN Graph, but every time inserting a new node, but instead of top-k greedy search from the closest node, we start from a random node → resulting in a small world structure.

Slide from Yusuke Matsui (CVPR 2020)

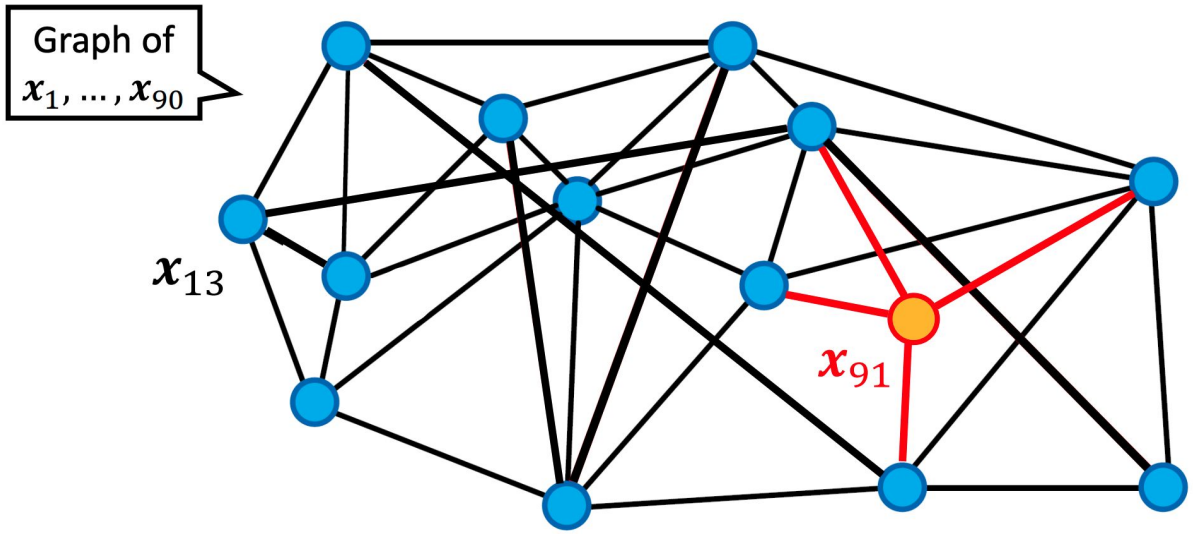
Navigable Small World (NSW)



- K-NN Graph, but every time inserting a new node, but instead of top-k greedy search from the closest node, we start from a random node → resulting in a small world structure.

Slide from Yusuke Matsui (CVPR 2020)

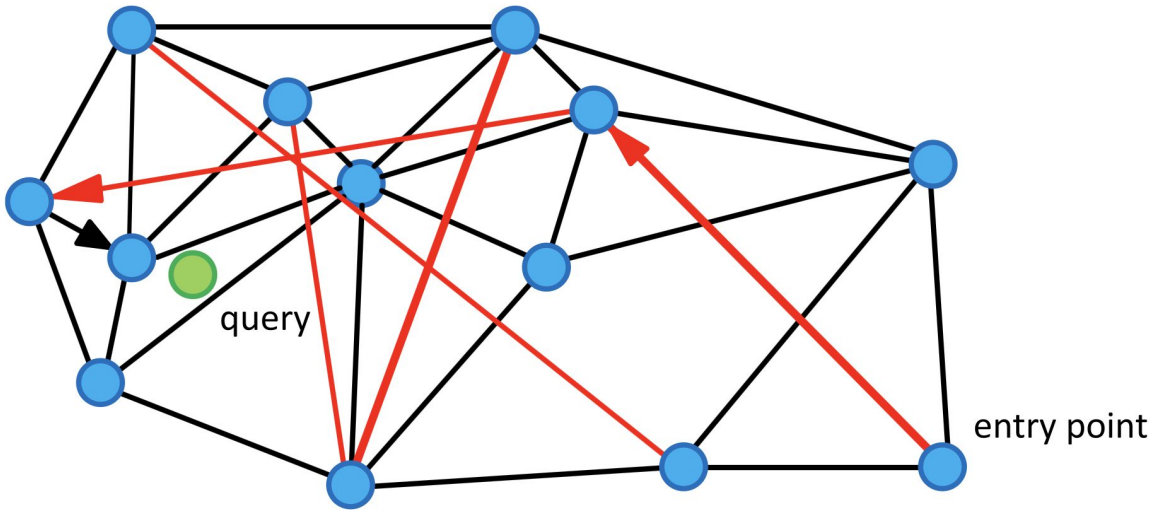
Navigable Small World (NSW)



- K-NN Graph, but every time inserting a new node, but instead of top-k greedy search in the closest neighborhood, we start from a random node → resulting in a small world structure.

Slide adapted from Yusuke Matsui (CVPR 2020)

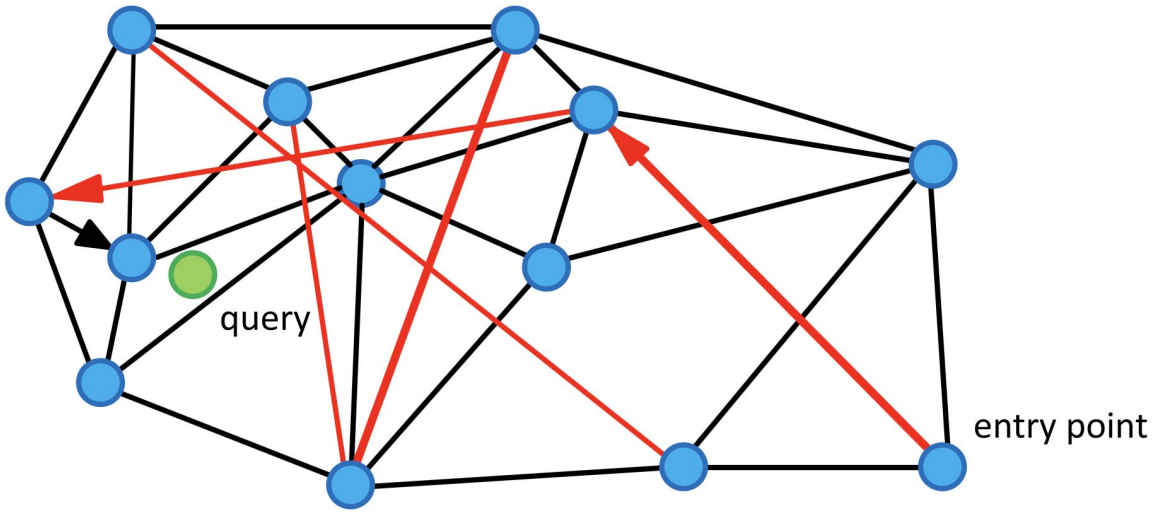
Navigable Small World (NSW)



- Given a query vector
- Start from a random entry point
- From the connected nodes, find the closest one to the query

[Malkov+, Information Systems, 2013]

Navigable Small World (NSW)



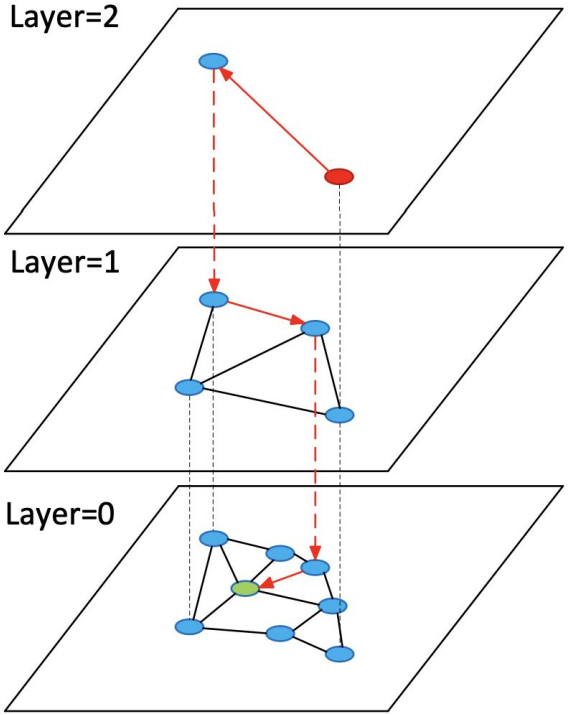
Observations:

- ➢ Long links reduce search path length. Search complexity (O(log^2(N))).
- ➢ You might want to start with nodes with higher degree!

[Malkov+, Information Systems, 2013]

Hierarchical Navigable Small World (HNSW)

- Adding hierarchy to NSW produces a graph where links are separated across different layers
- At the top layer, we have the longest links, and at the bottom layer, we have the shortest
- The closest neighbors from layer **l** becomes the entry points for layer **l-1**



Building HNSW Index

Vectors are **iteratively inserted one-by-one**.

Parameters:

- **M** - connections per node → recall vs memory usage
- **efConstruction** - breadth of search during insertion → recall vs build time

Complexity: O(log(N) * M * efConstruction) per vector → ~O(N*log(N)) in total

Building HNSW Index

Algorithm 1
INSERT(*hnsw*, *q*, *M*, M_{max}, *efConstruction*, m_L)
Input: multilayer graph *hnsw*, new element *q*, number of established connections *M*, maximum number of connections for each element per layer M_{max}, size of the dynamic candidate list *efConstruction*, normalization factor for level generation m_L
Output: update *hnsw* inserting element *q*

```
W ← ∅   // list for the currently found nearest elements
ep ← get enter point for hnsw
L ← level of ep   // top layer for hnsw
l ← ⌊-ln(unif(0..1))·mL⌋  // new element's level
for lc ← L … l+1
  W ← SEARCH-LAYER(q, ep, ef=1, lc)
  ep ← get the nearest element from W to q
for lc ← min(L, l) … 0
  W ← SEARCH-LAYER(q, ep, efConstruction, lc)
  neighbors ← SELECT-NEIGHBORS(q, W, M, lc) // alg. 3 or alg. 4
  add bidirectionall connectionts from neighbors to q at layer lc
  for each e ∈ neighbors   // shrink connections if needed
    eConn ← neighbourhood(e) at layer lc
    if |eConn| > Mmax // shrink connections of e
                      // if lc = 0 then Mmax = Mmax0
      eNewConn ← SELECT-NEIGHBORS(e, eConn, Mmax, lc)
                                          // alg. 3 or alg. 4
      set neighbourhood(e) at layer lc to eNewConn
  ep ← W
if l > L
  set enter point for hnsw to q
```

Inserting vector **q**

Select **insertion layer (l)** for q

Find the **entry point** to begin inserting q at its insertion layer

Connect q with its nearest neighbors on each layer from layer l to layer 0

Building HNSW Index

Algorithm 1
INSERT(*hnsw*, *q*, *M*, M_{max}, *efConstruction*, m_L)
Input: multilayer graph *hnsw*, new element *q*, number of established connections *M*, maximum number of connections for each element per layer M_{max}, size of the dynamic candidate list *efConstruction*, normalization factor for level generation m_L
Output: update *hnsw* inserting element *q*

```
W ← ∅    // list for the currently found nearest elements
ep ← get enter point for hnsw
L ← level of ep    // top layer for hnsw
l ← ⌊-ln(unif(0..1))·mL⌋  // new element's level
for lc ← L … l+1
  W ← SEARCH-LAYER(q, ep, ef=1, lc)
  ep ← get the nearest element from W to q
for lc ← min(L, l) … 0
  W ← SEARCH-LAYER(q, ep, efConstruction, lc)
  neighbors ← SELECT-NEIGHBORS(q, W, M, lc) // alg. 3 or alg. 4
  add bidirectionall connectionts from neighbors to q at layer lc
  for each e ∈ neighbors   // shrink connections if needed
    eConn ← neighbourhood(e) at layer lc
    if |eConn| > Mmax // shrink connections of e
                      // if lc = 0 then Mmax = Mmax0
      eNewConn ← SELECT-NEIGHBORS(e, eConn, Mmax, lc)
                                          // alg. 3 or alg. 4
      set neighbourhood(e) at layer lc to eNewConn
  ep ← W
if l > L
  set enter point for hnsw to q
```

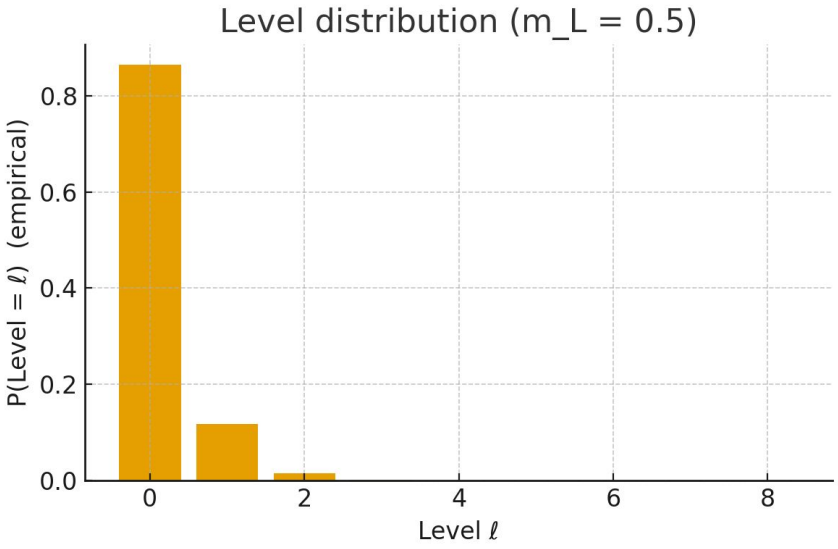
Inserting vector **q**

Select **insertion layer (l)** for q

Building HNSW Index

$$l \leftarrow \lfloor -\ln(unif(0..1)) \cdot m_L \rfloor$$

The number of vectors increases exponentially as we move down from higher layers to lower layers.



https://www.pinecone.io/learn/series/faiss/hnsw/

Building HNSW Index

Algorithm 1
INSERT(*hnsw*, *q*, *M*, M_{max}, *efConstruction*, m_L)
Input: multilayer graph *hnsw*, new element *q*, number of established connections *M*, maximum number of connections for each element per layer M_{max}, size of the dynamic candidate list *efConstruction*, normalization factor for level generation m_L
Output: update *hnsw* inserting element *q*

```
W ← ∅   // list for the currently found nearest elements
ep ← get enter point for hnsw
L ← level of ep   // top layer for hnsw
l ← ⌊-ln(unif(0..1))·mL⌋  // new element's level
for lc ← L … l+1
  W ← SEARCH-LAYER(q, ep, ef=1, lc)
  ep ← get the nearest element from W to q
for lc ← min(L, l) … 0
  W ← SEARCH-LAYER(q, ep, efConstruction, lc)
  neighbors ← SELECT-NEIGHBORS(q, W, M, lc) // alg. 3 or alg. 4
  add bidirectionall connectionts from neighbors to q at layer lc
  for each e ∈ neighbors   // shrink connections if needed
    eConn ← neighbourhood(e) at layer lc
    if |eConn| > Mmax // shrink connections of e
                      // if lc = 0 then Mmax = Mmax0
      eNewConn ← SELECT-NEIGHBORS(e, eConn, Mmax, lc)
                                          // alg. 3 or alg. 4
      set neighbourhood(e) at layer lc to eNewConn
  ep ← W
if l > L
  set enter point for hnsw to q
```

Inserting vector **q**

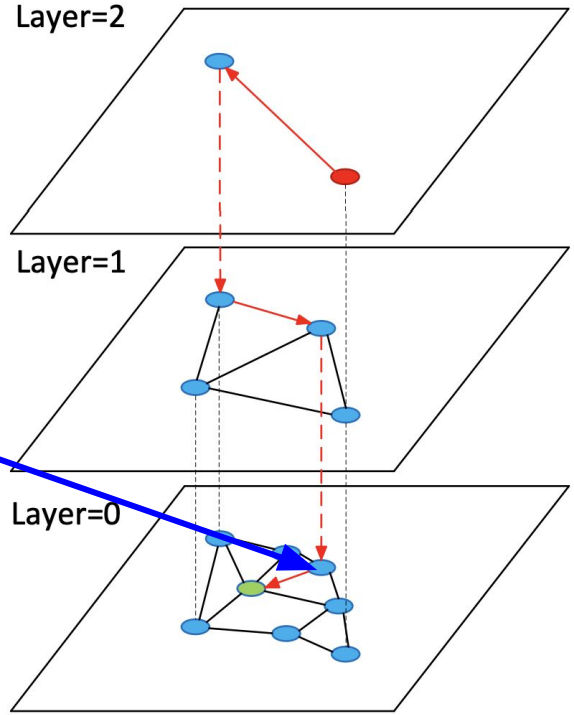
~~Select **insertion layer (l)** for q~~

Find the **entry point** to begin inserting q at its insertion layer

Building HNSW Index

Inserting vector **q** (insertion layer l = 0)

Find the entry point at layer 0



Building HNSW Index

Algorithm 1
INSERT(*hnsw*, *q*, *M*, M_{max}, *efConstruction*, m_L)
Input: multilayer graph *hnsw*, new element *q*, number of established connections *M*, maximum number of connections for each element per layer M_{max}, size of the dynamic candidate list *efConstruction*, normalization factor for level generation m_L
Output: update *hnsw* inserting element *q*

```
W ← ∅   // list for the currently found nearest elements
ep ← get enter point for hnsw
L ← level of ep   // top layer for hnsw
l ← ⌊-ln(unif(0..1))·mL⌋  // new element's level
for lc ← L … l+1
  W ← SEARCH-LAYER(q, ep, ef=1, lc)
  ep ← get the nearest element from W to q
for lc ← min(L, l) … 0
  W ← SEARCH-LAYER(q, ep, efConstruction, lc)
  neighbors ← SELECT-NEIGHBORS(q, W, M, lc) // alg. 3 or alg. 4
  add bidirectionall connectionts from neighbors to q at layer lc
  for each e ∈ neighbors   // shrink connections if needed
    eConn ← neighbourhood(e) at layer lc
    if |eConn| > Mmax // shrink connections of e
                      // if lc = 0 then Mmax = Mmax0
      eNewConn ← SELECT-NEIGHBORS(e, eConn, Mmax, lc)
                                          // alg. 3 or alg. 4
      set neighbourhood(e) at layer lc to eNewConn
  ep ← W
if l > L
  set enter point for hnsw to q
```

Inserting vector **q**

~~Select **insertion layer (l)** for q~~

~~Find the **entry point** to begin inserting q at its insertion layer~~

Building HNSW Index

Algorithm 1
INSERT(*hnsw*, *q*, *M*, M_{max}, *efConstruction*, m_L)
Input: multilayer graph *hnsw*, new element *q*, number of established connections *M*, maximum number of connections for each element per layer M_{max}, size of the dynamic candidate list *efConstruction*, normalization factor for level generation m_L
Output: update *hnsw* inserting element *q*

```
W ← ∅   // list for the currently found nearest elements
ep ← get enter point for hnsw
L ← level of ep   // top layer for hnsw
l ← ⌊-ln(unif(0..1))·mL⌋  // new element's level
for lc ← L … l+1
  W ← SEARCH-LAYER(q, ep, ef=1, lc)
  ep ← get the nearest element from W to q
for lc ← min(L, l) … 0
  W ← SEARCH-LAYER(q, ep, efConstruction, lc)
  neighbors ← SELECT-NEIGHBORS(q, W, M, lc) // alg. 3 or alg. 4
  add bidirectionall connectionts from neighbors to q at layer lc
  for each e ∈ neighbors   // shrink connections if needed
    eConn ← neighbourhood(e) at layer lc
    if |eConn| > Mmax // shrink connections of e
                      // if lc = 0 then Mmax = Mmax0
      eNewConn ← SELECT-NEIGHBORS(e, eConn, Mmax, lc)
                                          // alg. 3 or alg. 4
      set neighbourhood(e) at layer lc to eNewConn
  ep ← W
if l > L
  set enter point for hnsw to q
```

Inserting vector **q**

~~Select **insertion layer (l)** for q~~

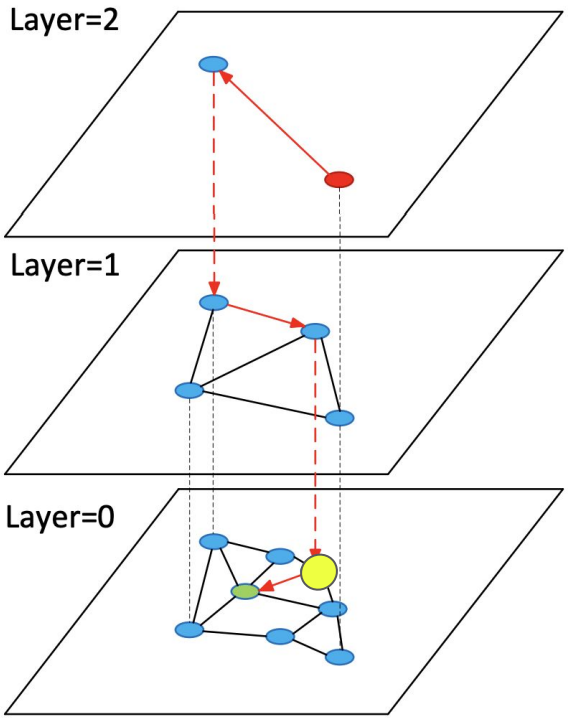
~~Find the **entry point** to begin inserting q at its insertion layer~~

Connect q with its nearest neighbors on each layer from layer l to layer 0

Building HNSW Index

We are at layer 0:

- Start traversing from the entry point (ep)
- **SEARCH_LAYER**: Find **ef** closest nodes to **q**
- **SELECT_NEIGHBORS**: Select M out of ef nodes as neighbors for q
    - If one of q's neighbors (say q2) have too many connections, we re-select neighbors for q2



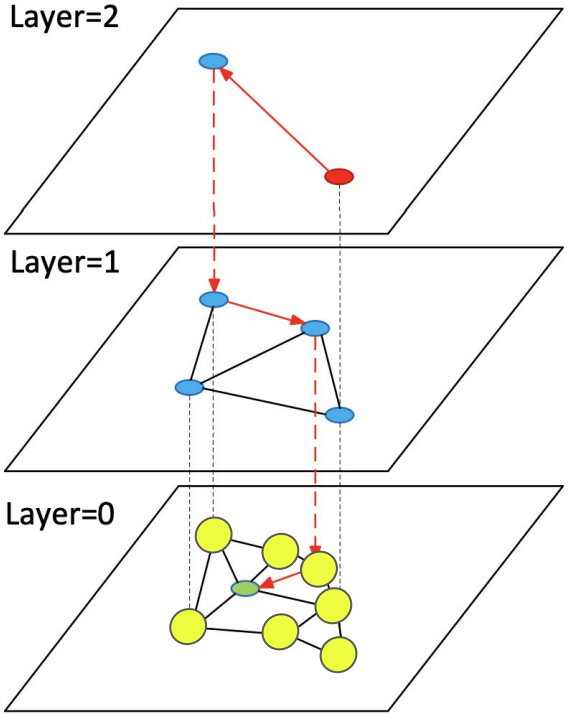
Building HNSW Index

Algorithm 2
SEARCH-LAYER(q, ep, ef, l_c)
Input: query element q, enter points ep, number of nearest to q elements to return ef, layer number l_c
Output: ef closest neighbors to q

```
v ← ep      // set of visited elements
C ← ep      // set of candidates
W ← ep      // dynamic list of found nearest neighbors
while |C| > 0
  c ← extract nearest element from C to q
  f ← get furthest element from W to q
  if distance(c, q) > distance(f, q)
    break   // all elements in W are evaluated
  for each e ∈ neighbourhood(c) at layer l_c   // update C and W
    if e ∉ v
      v ← v ∪ e
      f ← get furthest element from W to q
      if distance(e, q) < distance(f, q) or |W| < ef
        C ← C ∪ e
        W ← W ∪ e
        if |W| > ef
          remove furthest element from W to q
return W
```

- Find the top ef neighbor in the current layer (W)
- Always **expand the currently closest node** to query

Building HNSW Index



Building HNSW Index

Algorithm 3
SELECT-NEIGHBORS-SIMPLE(q, C, M)
Input: base element q, candidate elements C, number of neighbors to return M
Output: M nearest elements to q
return M nearest elements from C to q

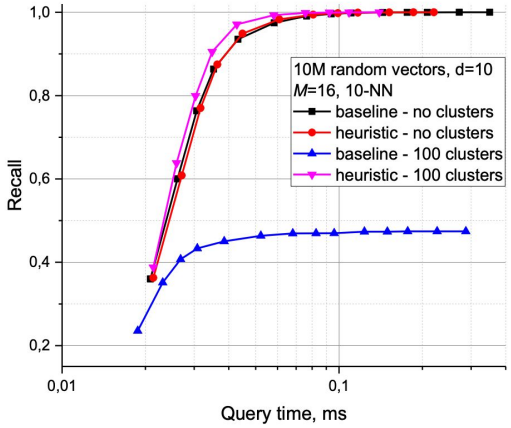


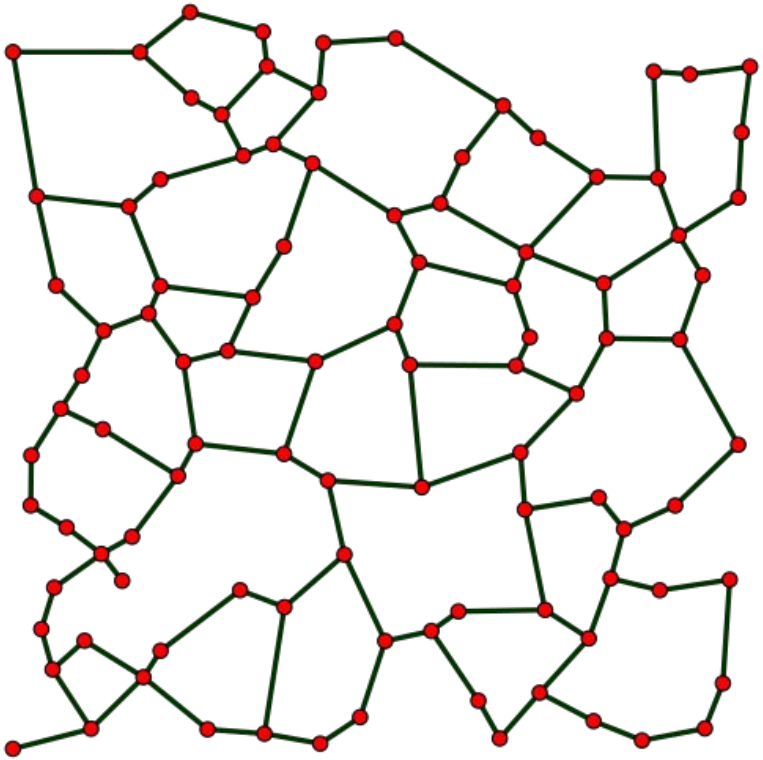
Fig. 7. Effect of the method of neighbor selections (baseline corresponds to alg. 3, heuristic to alg. 4) on clustered (100 random isolated clusters) and non-clustered d=10 random vector data.

HSNW neighbor selection procedure

Algorithm 4
SELECT-NEIGHBORS-HEURISTIC(q, C, M, l_c, *extendCandidates*, *keepPrunedConnections*)
Input: base element q, candidate elements C, number of neighbors to return M, layer number l_c, flag indicating whether or not to extend candidate list *extendCandidates*, flag indicating whether or not to add discarded elements *keepPrunedConnections*
Output: M elements selected by the heuristic

```
R ← ∅
W ← C   // working queue for the candidates
if extendCandidates   // extend candidates by their neighbors
  for each e ∈ C
    for each e_adj ∈ neighbourhood(e) at layer l_c
      if e_adj ∉ W
        W ← W ∪ e_adj
W_d ← ∅   // queue for the discarded candidates
while |W| > 0 and |R| < M
  e ← extract nearest element from W to q
  if e is closer to q compared to any element from R
    R ← R ∪ e
  else
    W_d ← W_d ∪ e
if keepPrunedConnections  // add some of the discarded
                          // connections from W_d
  while |W_d| > 0 and |R| < M
    R ← R ∪ extract nearest element from W_d to q
return R
```

Building HNSW Index



Relative Neighborhood Graph (RNG)

Naive k-NN rule for neighbor selection leads to dense graph. The search might get stuck in a cluster. HNSW use a heuristic to avoid this.

Relative neighborhood graph: “Connect A and B only if there’s no third point C that is closer to both A and B than A and B are to each other.”

Building HNSW Index

Algorithm 1
INSERT(*hnsw*, *q*, *M*, M_{max}, *efConstruction*, m_L)
Input: multilayer graph *hnsw*, new element *q*, number of established connections *M*, maximum number of connections for each element per layer M_{max}, size of the dynamic candidate list *efConstruction*, normalization factor for level generation m_L
Output: update *hnsw* inserting element *q*

```
W ← ∅   // list for the currently found nearest elements
ep ← get enter point for hnsw
L ← level of ep   // top layer for hnsw
l ← ⌊-ln(unif(0..1))·mL⌋  // new element's level
for lc ← L … l+1
  W ← SEARCH-LAYER(q, ep, ef=1, lc)
  ep ← get the nearest element from W to q
for lc ← min(L, l) … 0
  W ← SEARCH-LAYER(q, ep, efConstruction, lc)
  neighbors ← SELECT-NEIGHBORS(q, W, M, lc) // alg. 3 or alg. 4
  add bidirectionall connectionts from neighbors to q at layer lc
  for each e ∈ neighbors   // shrink connections if needed
    eConn ← neighbourhood(e) at layer lc
    if |eConn| > Mmax // shrink connections of e
                      // if lc = 0 then Mmax = Mmax0
      eNewConn ← SELECT-NEIGHBORS(e, eConn, Mmax, lc)
                                          // alg. 3 or alg. 4
      set neighbourhood(e) at layer lc to eNewConn
  ep ← W
if l > L
  set enter point for hnsw to q
```

Inserting vector **q**

~~Select **insertion layer (l)** for q~~

~~Find the **entry point** to begin inserting q at its insertion layer~~

~~Connect q with its nearest neighbors on each layer from layer l to layer 0~~

Building HNSW Index

Vectors are **iteratively inserted one-by-one**.

Parameters:

- **M** - connections per node → recall vs memory usage
- **efConstruction** - breadth of search during insertion → recall vs build time

Complexity: O(log(N) * M * efConstruction) per vector → ~O(N*log(N)) in total

Query HNSW Index

Parameters:

- **efSearch** - breadth of search → recall vs runtime

Complexity: O(logN * M * efSearch)

Query HNSW Index

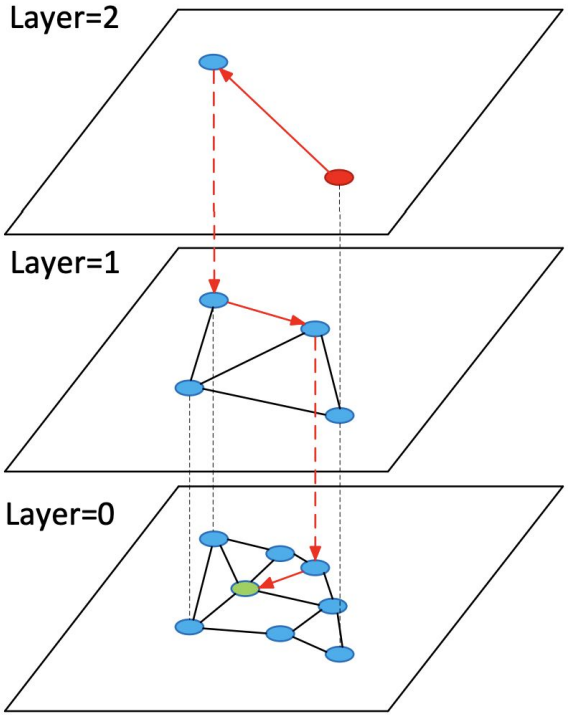
Algorithm 5

K-NN-SEARCH(*hnsw*, *q*, *K*, *ef*)

Input: multilayer graph *hnsw*, query element *q*, number of nearest neighbors to return *K*, size of the dynamic candidate list *ef*

Output: *K* nearest elements to *q*

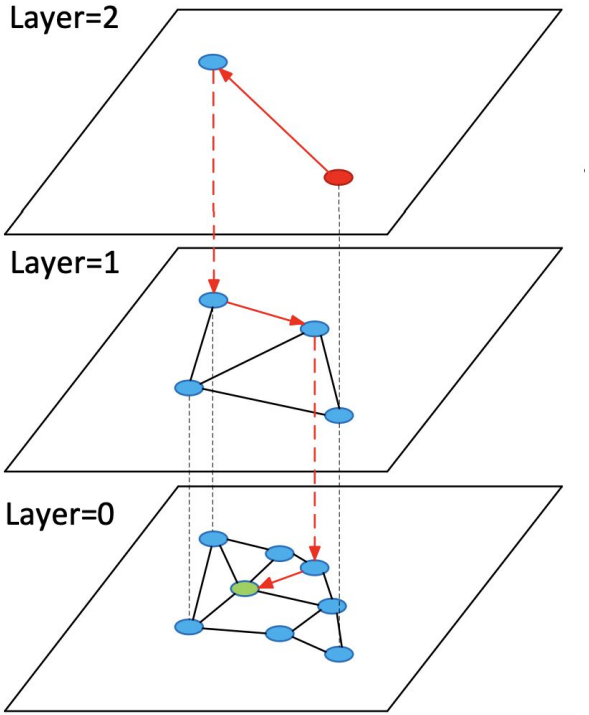
```
W ← ∅   // set for the current nearest elements
ep ← get enter point for hnsw
L ← level of ep    // top layer for hnsw
for lc ← L … 1
    W ← SEARCH-LAYER(q, ep, ef=1, lc)
    ep ← get nearest element from W to q
W ← SEARCH-LAYER(q, ep, ef, lc =0)
return K nearest elements from W to q
```



Query HNSW Index

From layer L → 1: Greedy search, choose the local nearest neighbor as the entry point for the next layer

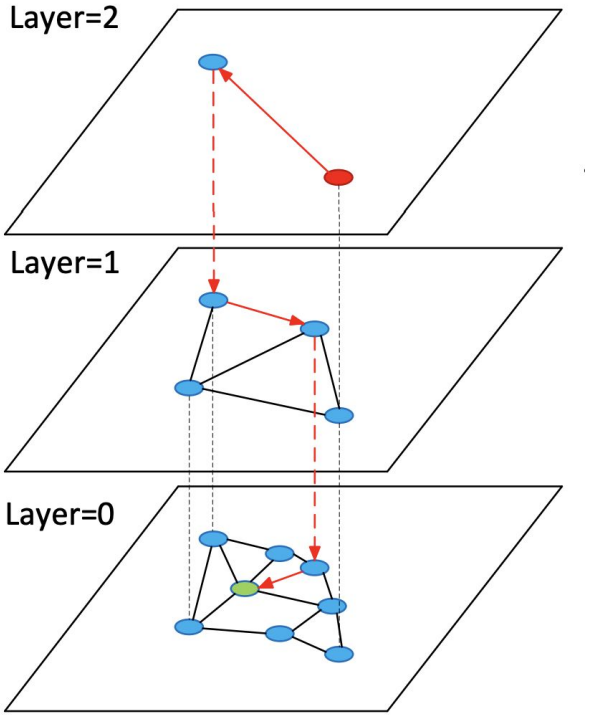
At layer 0: Search for the K nearest elements



Maintain HNSW Index

Insertion: Covered in build index

Deletion: Mark node as deleted and keep the edges



Indexing 1M 256D vectors with HNSW vs IVF (Cluster Index)

testing HNSW Flat

efSearch 16	0.011 ms per query, R@1 0.8740
efSearch 32	0.020 ms per query, R@1 0.9492
efSearch 64	0.033 ms per query, R@1 0.9779
efSearch 128	0.059 ms per query, R@1 0.9887
efSearch 256	0.104 ms per query, R@1 0.9920

testing IVF Flat (baseline)

nprobe 1	0.076 ms per query, R@1 0.4085
nprobe 4	0.067 ms per query, R@1 0.6331
nprobe 16	0.078 ms per query, R@1 0.8263
nprobe 64	0.141 ms per query, R@1 0.9470
nprobe 256	0.344 ms per query, R@1 0.9861

https://github.com/facebookresearch/faiss/wiki/Indexing-1M-vectors

Libraries that implemented HNSW

FAISS
Scalable Search With Facebook AI

APACHE LUCENE™

hnswlib Public

Hierarchical Navigable Small World (HNSW)

- Fast, high-recall search with support for incremental insertions
- Uses a lot of memory, but if **memory isn't a constraint, HNSW is the preferred choice for Approximate Nearest Neighbor Search**